

Literatuur (per hoofdstuk)

Literatuur

Volgende boeken zijn aangeraden als algemene achtergrondinformatie bij de cursus. Ze zijn allen beschikbaar in de bibliotheek van de UA.

- [Ghez02] Fundamentals of Software Engineering, C. Ghezzi, M. Jazayeri, D. Mandroli, 2nd edition, Prentice Hall, 2002.
 - Timeless because of its emphasis on principles, but a bit difficult.
- [Pres00] Software Engineering - A Practitioner's Approach, R. Pressman, Mc-Graw Hill, Fifth Edn., 2000.
 - Very practical and quite deep, but contains a lot of material and is therefore quite thick and a bit expensive.
- [Somm05] Software Engineering, I. Sommerville, Addison-Wesley, Ninth Edn., 2011
 - Very popular and broad plus reads quite easily, but often lacks profoundness.

Extra information on SCRUM; the process you will use for the Capstone Project

- [Rubi13] Essential Scrum: A Practical Guide to the most popular agile process. Kenneth S. Rubin. Addison-Wesley, 2013

Introduction (UML)

- UML Modeling Language -- Notation Guide, version 1.3
 - [<http://www.uml.org/> (<http://www.uml.org/>)]
 - The specification of the UML notation.
- "Iterative and Incremental Development: A Brief History" Craig Larman and Victor R. Basili, IEEE Computer, June 2003
 - [<http://www.craigarman.com/wiki/downloads/misc/history-of-iterative-larman-and-basili-ieee-computer.pdf> (<http://www.craigarman.com/wiki/downloads/misc/history-of-iterative-larman-and-basili-ieee-computer.pdf>)]
 - A historical overview on how the mindset of software projects changed from waterfall to iterative and incremental development.
- "The Five Orders of Ignorance" (Phillip G. Armour), Communications of the ACM (October 2000)
 - [<http://dx.doi.org/10.1145/352183.352194> (<http://dx.doi.org/10.1145/352183.352194>)]
- How Many Millions of Lines of Code Does It Take? [<https://www.visualcapitalist.com/millions-lines-of-code/> (<https://www.visualcapitalist.com/millions-lines-of-code/>)]
 - How many millions of lines of code does it take to make the modern program, web service, car, or airplane possible? The range is extraordinary: the average iPhone app has less than 50,000 lines of code, while Google's entire code base is two billion lines for all services.

- Apollo's computer "eventually required about 145,000 lines of code in all, compared with about 62 million lines of code required today to operate Facebook and more than two billion lines of code for Google," according to the Journal.
- [<https://fedtechmagazine.com/article/2019/07/tech-behind-apollo-11s-guidance-computer> (<https://fedtechmagazine.com/article/2019/07/tech-behind-apollo-11s-guidance-computer>)]
- The code is on github [<https://github.com/chrislgarry/Apollo-11> (<https://github.com/chrislgarry/Apollo-11>).]

Requirements

- A dilbert cartoon about requirements and negotiating with customers
- Applying Use Cases - a Practical Guide, Geri Schneider, Jason, P. Winters, Addison-Wesley, 1998.
 - An easy to read an practical guide on how to iteratively develop a set of use cases and how to exploit it for project planning.
- Object-Oriented Software Engineering: A Use-Case Driven Approach, I. Jacobson et. al., Addison-Wesley, 1992.
 - The book introducing use cases. Also tells how to use them during a complete software project.
- "User stories as lightweight requirements for agile clinical decision support development", Vaishnavi Kannan, Mujeeb A Basit, Puneet Bajaj, Angela R Carrington, Irma B Donahue, Emily L Flahaven, Richard Medford, Tsedey Melaku, Brett A Moran, Luis E Saldana. In Journal of the American Medical Informatics Association, Volume 26, Issue 11, November 2019, Pages 1344–1354
 - [<https://doi.org/10.1093/jamia/ocz123> (<https://doi.org/10.1093/jamia/ocz123>)]
 - An experience report applying user stories in combination with use case diagrams to manage the requirements in a health care application. In healthcare, human lives might be affected, so explicit requirements play an important role.
- "Structuring Use Cases with Goals", Alister Cockburn, Journal of Object-Oriented Programming Sept-Oct & Nov-Dec 1997.
 - [https://www.researchgate.net/publication/2807676_Structuring_Use_cases_with_goals (https://www.researchgate.net/publication/2807676_Structuring_Use_cases_with_goals).]
 - Very popular article giving concrete and practical guidelines concerning use-cases.
- "Misuse Cases: Use Cases with Hostile Intent" Ian Alexander, in IEEE Software, vol. 20, no. 01, pp. 58-66, 2003. doi: 10.1109/MS.2003.1159030
 - [<https://doi.org/10.1109/MS.2003.1159030> (<https://doi.org/10.1109/MS.2003.1159030>)]
 - Explains a negative form of a use case; a negative scenario. Use- and misuse-case diagrams are valuable in threat and hazard analysis.
- "Discovering, Analyzing, and Managing Safety Stories in Agile Projects," Jane Cleland-Huang and Michael Vierhauser, 2018 IEEE 26th International Requirements Engineering Conference (RE), 2018, pp. 262-273, doi: 10.1109/RE.2018.00034.
 - [<https://doi.org/10.1109/RE.2018.00034> (<https://doi.org/10.1109/RE.2018.00034>)]
 - Addresses the specific problems of discovering, analyzing, specifying, and managing safety requirements within the agile Scrum process.

Software Architecture

- Pattern-Oriented Software Architecture: A System of Patterns, Frank Buschman, Regine Meunier, Hans Rohnert, Peter Somerlad, Michael Stal, Wiley and Sons, 1996.
 - Introduces architectural styles in pattern form. Also covers some design patterns and idioms.
- Design Patterns: Elements of Reusable Object-Oriented Software, Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides, Addison-Wesley, 1995.
 - The classic. At design level but also viewed as micro-architecture.
- Software architecture: perspectives on an emerging discipline, Mary Shaw, David Garlan, Prentice-Hall, 1996.
 - The book introducing software architecture.
- Software architecture in practice, Len Bass, Paul Clements, Rick Kazman, Addison-Wesley, 1998.
 - A very deep and practical treatment of software architecture.
- "The 4+1 View Model of Architecture", Philippe Kruchten, IEEE Software, November 1995 (Vol. 12, No. 6) pp. 42-50.
 - [<https://doi.ieeecomputersociety.org/10.1109/52.469759>
(<https://doi.ieeecomputersociety.org/10.1109/52.469759>)]
 - A paper that illustrates convincingly the need for various perspectives on the design of a system.

Project Management

- Software Project Management, B. Hughes and M. Cotterell, McGraw Hill, 2006. (4th Edition)
 - Good practical examples on PERT, Gantt, Time-sheets, ...
- "Using Belbin's leadership role to improve team effectiveness: An empirical investigation". Sallie M. Henry, K. Todd Stevens, Journal of Systems and Software, Volume 44, Issue 3, January 1999, Pages 241-250, ISSN 0164-1212.
 - [[https://dx.doi.org/10.1016/S0164-1212\(98\)10060-2](https://dx.doi.org/10.1016/S0164-1212(98)10060-2) ([https://dx.doi.org/10.1016/S0164-1212\(98\)10060-2](https://dx.doi.org/10.1016/S0164-1212(98)10060-2))]
 - Illustrates that even for student projects doing software engineering it is good to cover the important roles (especially leadership) in a team.
 - Several on-line tests existed to determine your preferred role in a team. However, they are deemed inappropriate, even illegal.
- "All Late Projects Are the Same". Tom De Marco, IEEE Software, pp. 102-103, November/December, 2011
 - [<https://doi.ieeecomputersociety.org/10.1109/MS.2011.134>
(<https://doi.ieeecomputersociety.org/10.1109/MS.2011.134>)]
 - All projects that finish late have this one thing in common ... they started late.

Design by Contract

- Object-Oriented Software Construction, B. Meyer, Prentice Hall, Second Edn., 1997.
 - The book introducing Design by Contract. Also an excellent treatment on the do's and don'ts of object-oriented development.
- "Put it in the contract: The lessons of Ariane", Jean-Marc Jézéquel and Bertrand Meyer, IEEE Computer, January 1997, Vol30, no. 2, pages 129-130.

- [<https://www.irisa.fr/pampa/EPEE/Ariane5.html>
(<https://www.irisa.fr/pampa/EPEE/Ariane5.html>).]
- A (heatedly debated) column arguing that Design by Contract would have prevented the crash of the first Ariane5 missile.
- "Critique of 'Put it in the contract: The lessons of Ariane'", Ken Garlington. Technical paper.
 - The original version [<http://home.flash.net/~kennieg/ariane.html>
(<http://home.flash.net/%7Ekennieg/ariane.html>).] is lost. There is still a copy at Scribd; do not register there [<https://www.scribd.com/document/59743067/SQE-Lec-02-Critique-of-Design-by-Contract> (<https://www.scribd.com/document/59743067/SQE-Lec-02-Critique-of-Design-by-Contract>)]
 - An article providing counterarguments to the former. Yet by doing so gives an excellent comparison with Testing and Code Inspection.
- "Simplifying Microservice testing with Pacts", Ron Holshausen.
 - [<https://dius.com.au/2014/05/19/simplifying-micro-service-testing-with-pacts/>
(<https://dius.com.au/2014/05/19/simplifying-micro-service-testing-with-pacts/>).]
 - Contract testing is the primary way to ensure that systems work as expected for micro-services.
- "Programming with Assertions", SUN
 - [<https://docs.oracle.com/javase/7/docs/technotes/guides/language/assert.html>
(<https://docs.oracle.com/javase/7/docs/technotes/guides/language/assert.html>).]
 - An article discussing how to use ASSERTs, one of the new features since Java 1.4. Among others, it confirms that Java-assertions are too weak for a generic "design by contract".
- "iContract: Design by Contract in Java", Oliver Ensling
 - [<https://www.infoworld.com/article/2074956/icontract-design-by-contract-in-java.html>
(<https://www.infoworld.com/article/2074956/icontract-design-by-contract-in-java.html>).]
 - An article (including java sources) discussing a tool for contracts in Java.

Testing

- "A Survey of Unit Testing Practices", Per Runeson
 - [<https://dx.doi.org/10.1109/MS.2006.91> (<https://dx.doi.org/10.1109/MS.2006.91>).]
 - Discusses the state-of-the-practice with unit testing. A nice quote "Most respondents said that running all unit tests took just seconds or minutes, but some respondents had unit test suites that took hours to execute".
- "Test Fatigue", Gerard J. Holzman. in IEEE Software, vol. 37, no. 4, pp. 11-16, July-Aug. 2020
 - [<https://doi.org/10.1109/MS.2020.2986107> (<https://doi.org/10.1109/MS.2020.2986107>).]
 - Argues that the more rigorous testing you do, the more problems you will find. Thus it is tempting to test less ... or not?

Formal Specifications

- D. Cofer et al., "A Formal Approach to Constructing Secure Air Vehicle Software," in Computer, vol. 51, no. 11, pp. 14-23, Nov. 2018
 - Use of formal methods as part of an ethical hack (an actual cyber attack) on a military aircraft during flight.
 - [<https://doi.org/10.1109/MC.2018.2876051> (<https://doi.org/10.1109/MC.2018.2876051>).]

- J. Kapinski, J. V. Deshmukh, X. Jin, H. Ito and K. Butts, "Simulation-Based Approaches for Verification of Embedded Control Systems: An Overview of Traditional and Advanced Modeling, Testing, and Verification Techniques," in IEEE Control Systems Magazine, vol. 36, no. 6, pp. 45-64, Dec. 2016
 - [<https://doi.org/10.1109/MCS.2016.2602089> (<https://doi.org/10.1109/MCS.2016.2602089>)]
 - A comprehensive overview of techniques for safety-critical systems. Explains well how formal methods complement simulation and testing.
- A HTML page with errors
 - A sample page used to start up the lecture.



formalspecN.html

Domain Models

- The software specification cartoon
 - [[BlackAndWhiteVersion](https://blogger.googleusercontent.com/img/b/R29vZ2xl/AVvXsEgLIYoSRAXzI5fJrY94hXDPckyjGALTk5f0SgBlgcTEvwUmaWlau8ZCul4ND05QuafbMlnZovbB4nDsRdCnrjtJuX6aTJasP5dH81o2hXPwA05UfSCZlcSirXSHiCH8I7z3QLC/s1600/SoftwareSpecCartoon.gif) (<https://blogger.googleusercontent.com/img/b/R29vZ2xl/AVvXsEgLIYoSRAXzI5fJrY94hXDPckyjGALTk5f0SgBlgcTEvwUmaWlau8ZCul4ND05QuafbMlnZovbB4nDsRdCnrjtJuX6aTJasP5dH81o2hXPwA05UfSCZlcSirXSHiCH8I7z3QLC/s1600/SoftwareSpecCartoon.gif>) | [ColouredVersion](https://www.reddit.com/media?url=https%3A%2F%2Fexternal-preview.redd.it%2Ftj4LJ7pPUQLr_IFbOi7nT097Bao5tGrtWf7QqIWlH0.jpg%3Fwidth%3D640%26crop%3Dsmart%26auto%3Dwebp%26s%3Dbe0193c34ffee4a4f7d57ee531885ca7ad865e5b) (https://www.reddit.com/media?url=https%3A%2F%2Fexternal-preview.redd.it%2Ftj4LJ7pPUQLr_IFbOi7nT097Bao5tGrtWf7QqIWlH0.jpg%3Fwidth%3D640%26crop%3Dsmart%26auto%3Dwebp%26s%3Dbe0193c34ffee4a4f7d57ee531885ca7ad865e5b)]
 - One of the earlier jokes on the various stages of a software project. Numerous versions circulate on the internet; the following give just a flavour of what's out there.
- "How the Boeing 737 Max Disaster Looks to a Software Developer" Gregory Travis. IEEE Spectrum, April 18.
 - [<https://spectrum.ieee.org/aerospace/aviation/how-the-boeing-737-max-disaster-looks-to-a-software-developer> (<https://spectrum.ieee.org/aerospace/aviation/how-the-boeing-737-max-disaster-looks-to-a-software-developer>)]
- "The Boeing 737 MAX: Lessons for Engineering Ethics." Herkert, J., Borenstein, J. & Miller, K. Science and Engineering Ethics (2020).
 - [<https://doi.org/10.1007/s11948-020-00252-y> (<https://doi.org/10.1007/s11948-020-00252-y>)]
- Product Line Hall of Fame
 - [<http://splc.net/fame.html> (<http://splc.net/fame.html>)]
 - A list of software companies who organised their software production process according to a software-product-lines approach

Quality Control

- Case Study – Denver International Airport Baggage Handling System – An illustration of ineffectual decision making
 - [https://calleam.com/WTPF/?page_id=2086 (https://calleam.com/WTPF/?page_id=2086) | <https://calleam.com/WTPF/wp-content/uploads/articles/DIABaggage.pdf> (<https://calleam.com/WTPF/wp-content/uploads/articles/DIABaggage.pdf>).]
 - A report on one of the best-known examples of how modern software development may screw up.
- Case Study - FBI Sentinel System
 - [<https://doi.ieeecomputersociety.org/10.1109/MC.2012.2> (<https://doi.ieeecomputersociety.org/10.1109/MC.2012.2>).]
 - Why the FBI Can't Build a Case Management System (Jerome W. Israel). IEEE Software, June 2012
 - A report illustrating why a seemingly innocent project (a database of FBI case files) can become overly complex if not handled properly. Several important lessons learned are deduced.
- Tacoma Narrows Bridge Disaster
 - [https://en.wikipedia.org/wiki/Tacoma_Narrows_Bridge (https://en.wikipedia.org/wiki/Tacoma_Narrows_Bridge).]
 - We are not the only ones who screw up; this is the famous example of the collapsing bridge.

Software Metrics

- The weather on D-Day
 - [<http://medium.com/history-and-politics/85ea0491a14f> (<http://medium.com/history-and-politics/85ea0491a14f>).]
 - Illustrating how crucial a good estimation might be
- Software Metrics: A rigorous & Practical Approach, Norman E. Fenton, Shari I. Pfleeger, Thompson Computer Press, 1996.
 - Thorough treatment of metrics theory with lots of advice to make it digestible.
- Fingers in the air: a Gentle Introduction to Software Estimation
 - [<http://www.methodsandtools.com/archive/archive.php?id=79> (<http://www.methodsandtools.com/archive/archive.php?id=79>).]
 - An article on the distinction between estimates, targets and commitments, incl. the famous "cone of uncertainty".
- Software Metrics (Patti Beadles) [rec.humor.funny]
 - [<https://www.netfunny.com/rhf/jokes/93q4/metrics.html> (<https://www.netfunny.com/rhf/jokes/93q4/metrics.html>).]
 - A humorous list of software metrics.
- A cartoon on measuring the output of Quality reviews using wtf / minute
 - [<https://www.osnews.com/images/comics/wtfm.jpg> (<https://www.osnews.com/images/comics/wtfm.jpg>).]
- Estimating With Use Case Points

- [<https://www.methodsandtools.com/archive/archive.php?id=25> (<https://www.methodsandtools.com/archive/archive.php?id=25>).]
- An article on use-case points, as a way to quantify the size and complexity of a requirement specification

Refactoring

- Refactoring, Improving the Design of Existing Code by Martin Fowler, Addison-Wesley, 1999.
 - A practical book explaining when and how to use refactorings to cure some typical code-smells.
- Object-Oriented Reengineering Patterns by Serge Demeyer, Stéphane Ducasse and Oscar Nierstrasz, Morgan Kaufmann, 2002.
 - A book describing how one can reengineer object-oriented legacy systems.
- Code smells
 - [<http://c2.com/cgi/wiki?CodeSmell> (<http://c2.com/cgi/wiki?CodeSmell>)]
 - Wiki-web with discussion on code smells, i.e. symptoms of poorly designed code.
- Refactoring
 - [<http://www.refactoring.com/> (<http://www.refactoring.com/>)]
 - refactoring web-site discussing solutions for the above code-smells.
- "Big Ball of Mud", Brian Foote and Joseph Yoder; Fourth Conference on Patterns Languages of Programs (PLoP '97/EuroPLoP '97)
 - [<http://www.laputan.org/mud/mud.html> (<http://www.laputan.org/mud/mud.html>)]
 - Quite a funny read, illustrating very well how software systems degrade into poor designs.

Conclusion (Ethics and Professional Practice)

- Software Engineering Code of Ethics and Professional Practice
 - [<https://ethics.acm.org/code-of-ethics/software-engineering-code/> (<https://ethics.acm.org/code-of-ethics/software-engineering-code/>).]
- VW Emissions scandal
 - [<http://calleam.com/WTPF/?p=7666> (<http://calleam.com/WTPF/?p=7666>)]
 - If you would work for Volkswagen and they ask you to install software on the car to cheat emission testing. What would you do?
- Facebook / Twitter API spying
 - [https://www.cjr.org/tow_center/facebook-twitter-api-restrictions.php (https://www.cjr.org/tow_center/facebook-twitter-api-restrictions.php)]
 - If your master thesis would study how social media affects elections, would you install spying software on the API of social media giants (facebook, twitter, ...)
- The 'Killer Robot' Case.
 - An especially relevant (and humorous !) series of newspaper articles describing an imaginary situation of a piece of software that kills people and how it could have been avoided. (Beware, the complete text is about 70 pages long; don't read it over a modem)
 - [<https://doi.org/10.18130/ey39-2c58> (<https://doi.org/10.18130/ey39-2c58>) | <https://onlineethics.org/cases/case-killer-robot> (<https://onlineethics.org/cases/case-killer-robot>).]
- The "No Silver Bullet" paper

- [<https://doi.org/10.1109/MC.1987.1663532> (<https://doi.org/10.1109/MC.1987.1663532>).]
- A very influential paper written by Fred Brooks, who in 1999 received the Turing Award for "landmark contributions to computer architecture, operating systems, and software engineering".